BEGINNER FRIENDLY · 2026

Data Engineering

A Clear Introduction for Freshers

From OLTP to the Medallion Architecture — explained with real examples & sample data

- OLTP & OLAP
- Data Warehouse · Data Lake · Lakehouse
- ETL & ELT
- Apache Spark · Data Pipelines
- Medallion Architecture

A self-study handbook · No prior experience required

What's Inside

Read it top to bottom — each topic builds on the previous one. Every concept comes with a plain-English analogy, a worked example, and sample data you can picture.

1. The Big Picture: What Is Data Engineering?

Imagine a large coffee-shop chain, **BrewBox**, with 500 stores. Every second, something happens: a customer buys a latte, an app user adds a loyalty point, a store runs low on milk. All of this creates **data**. On its own, that raw data is messy and scattered across dozens of systems. **Data engineering** is the discipline of collecting this data, cleaning it, organizing it, and delivering it to the people and tools that need it — reliably and on time.

> **Analogy — The city's water system.** Data engineers are the plumbers and civil engineers of data. Rain (raw data) falls everywhere. Engineers build reservoirs to store it, treatment plants to clean it, and pipes to carry clean water to every home (dashboards, reports, machine-learning models). You rarely think about the pipes — but nothing works without them.

A data engineer's job is to answer questions like: *Where does the data come from? How do we move it? Where do we store it? How do we make it clean and trustworthy? How do we serve it fast?* The rest of this guide walks through the core building blocks used to answer those questions.

The two worlds of data

Almost everything in data engineering starts with one fundamental split:

- **Operational systems (OLTP)** — run the day-to-day business. "Place the order. Charge the card. Update the stock."
- **Analytical systems (OLAP)** — help you understand the business. "Which product sold best last quarter? Which stores are shrinking?"

Understanding this split is the key that unlocks everything else, so we start there.

First, three flavours of data

Before we dive in, one more foundation. Data engineers deal with three kinds of data, and knowing the difference explains a lot about why warehouses, lakes, and lakehouses exist.

Type	What it means	Example	Fits neatly in a table?
Structured	Rows and columns with a fixed shape	A sales table; a bank statement	Yes — made for it
Semi-structured	Has some structure via tags/keys, but flexible	JSON, XML, CSV, log files	Partly — needs parsing
Unstructured	No predefined structure	Images, audio, video, free-text reviews	No — doesn't fit rows/columns

Table 1.1 — The three flavours of data, with everyday examples.

A traditional **warehouse** loves structured data. A **data lake** was invented precisely to hold semi-structured and unstructured data too. Keep this table in mind — it's the "why" behind several later sections.

2. OLTP — Online Transaction Processing

RUNS THE BUSINESS

OLTP stands for **Online Transaction Processing**. These are the systems that handle the live, everyday *transactions* of a business — the small, fast operations that happen constantly. When you swipe your card, book a seat, or add an item to a cart, an OLTP system is doing the work.

> **Key idea:** A **transaction** is a single, small unit of work — insert one order, update one balance, delete one record. OLTP systems are built to do **millions of these tiny operations per day**, each one fast and correct.

> **Analogy — The cashier at the counter.** An OLTP system is like the checkout till. It handles one customer at a time, very quickly: scan items, take payment, print receipt, next please. It is never asked "what were our total sales across all stores last year?" — that would hold up the queue. Its whole job is speedy, accurate, individual transactions.

What OLTP is good at

- **Fast reads and writes** of single records (milliseconds).
- **Many users at once** — thousands of people ordering simultaneously.
- **Data integrity** — it never charges you twice or loses your order. This is guaranteed by **ACID** properties (Atomicity, Consistency, Isolation, Durability).
- **Current data** — it stores what is true *right now*, not years of history.

Sample data — an OLTP orders table

Here is what a slice of BrewBox's live `orders` table might look like in its OLTP database. Notice: it is **normalized** — a customer is stored by ID, not by full details, to avoid repeating information.

order_id	customer_id	store_id	item	qty	amount	status	order_time
100871	C-4021	S-12	Latte	1	$4.50	PAID	2026-07-22 08:14:02
100872	C-3888	S-07	Cappuccino	2	$9.00	PAID	2026-07-22 08:14:05
100873	C-4021	S-12	Muffin	1	$3.25	PENDING	2026-07-22 08:14:09
100874	C-5567	S-31	Cold Brew	1	$5.00	PAID	2026-07-22 08:14:11

Table 2.1 — `orders` (OLTP): small, current, constantly changing rows.

A typical OLTP operation touches just **one row**: "insert order 100873," or "update order 100873 status to PAID." Simple, fast, done.

Common OLTP databases

MySQL, PostgreSQL, Oracle, Microsoft SQL Server, and cloud services like Amazon Aurora. These are usually **relational (SQL)** databases, though some businesses use NoSQL systems like MongoDB for OLTP too.

> **Why not just analyze data directly in OLTP?** Because a big analytical query — "sum every order for the last 3 years, grouped by product" — would scan millions of rows and **slow down the checkout for real customers**. Analytics needs a different kind of system. Enter OLAP.

3. OLAP — Online Analytical Processing

UNDERSTANDS THE BUSINESS

OLAP stands for **Online Analytical Processing**. These systems are built for *analysis*: looking across huge amounts of historical data to find patterns, trends, and totals. Instead of "process this one order," OLAP answers "how are we doing?"

> **Key idea:** OLAP reads **lots of rows** to produce **a few summary numbers**. Where OLTP writes small and often, OLAP reads big and periodically.

> **Analogy — The store manager's monthly review.** At month-end the manager sits down with every receipt from every store and asks big questions: "Which drink is our bestseller? Which region is growing? Are mornings busier than evenings?" She isn't serving customers — she's stepping back to see the whole picture. That reflective, aggregate view is OLAP.

What OLAP is good at

- **Aggregations** — sums, averages, counts across millions of rows.
- **Historical depth** — years of data kept for trend analysis.
- **Complex queries** — slice and dice by time, product, region, customer segment.
- **Read-heavy** — data is loaded in batches, then read many times; rarely updated row-by-row.

Sample data — an OLAP summary

OLAP data is often pre-summarized. Instead of individual orders, BrewBox's analytics system might hold monthly totals per product per region — perfect for a dashboard.

month	region	product	units_sold	revenue	avg_order_value
2026-06	North	Latte	48,230	$217,035	$4.50
2026-06	North	Cold Brew	19,540	$97,700	$5.00
2026-06	South	Latte	52,110	$234,495	$4.50
2026-06	South	Cappuccino	33,900	$152,550	$4.50

Table 3.1 — an OLAP-style aggregated `sales_summary` : many transactions rolled up into insights.

Behind each of those numbers sit tens of thousands of the tiny OLTP rows we saw earlier. OLAP's job was to **collect, group, and summarize** them.

The OLAP "cube"

Analysts often picture OLAP data as a **cube** with dimensions like *time*, *product*, and *region*. You can "slice" it (just June), "dice" it (June, North, Latte), "drill down" (from year → month → day), or "roll up" (from store → region → country). These operations make exploring data intuitive.

Common OLAP systems

Cloud data warehouses like **Snowflake, Google BigQuery, Amazon Redshift,** and **Azure Synapse**, plus tools like Databricks SQL. They use **columnar storage** (explained on the next page) to make big aggregations fast.

4. OLTP vs OLAP — Side by Side

This single comparison is worth memorizing. Almost every later concept exists to move data *from* the OLTP world *into* the OLAP world.

Aspect	OLTP (Transactional)	OLAP (Analytical)
Purpose	Run daily operations	Analyze & report
Typical question	"Charge this order."	"What were Q2 sales by region?"
Operations	Many small INSERT / UPDATE / DELETE	Large SELECT with GROUP BY
Rows touched per query	One or a few	Thousands to millions
Data age	Current state	Years of history
Users	Customers, apps (thousands live)	Analysts, managers (fewer)
Storage layout	Row-based	Column-based
Design	Normalized (no repetition)	Denormalized (built for reading)
Speed goal	Fast single writes	Fast big reads
Examples	MySQL, PostgreSQL, Oracle	Snowflake, BigQuery, Redshift

Table 4.1 — The core contrast every data engineer knows by heart.

Row storage vs column storage — why it matters

This is a favorite interview topic, and it's simpler than it sounds. Take four order rows with columns `id, product, amount`.

Stored on disk as…
1, Latte, 4.50
2, Muffin, 3.25
3, Latte, 4.50
4, Cold Brew, 5.00

Row storage (OLTP): each row kept together.

Stored on disk as…
ids: 1, 2, 3, 4
products: Latte, Muffin, Latte, Cold Brew
amounts: 4.50, 3.25, 4.50, 5.00

Column storage (OLAP): each column kept together.

If you want to fetch **one whole order** (OLTP), row storage wins — everything is in one place. If you want to **sum every amount** (OLAP), column storage wins — all the amounts sit together, so the system reads just that one column and skips the rest. This is why analytical warehouses store data by column: massive aggregations become dramatically faster.

Takeaway: OLTP and OLAP are not competitors — they are teammates. You need both. The whole "data platform" (warehouses, lakes, pipelines, Spark) exists to reliably move and reshape data from the OLTP side to the OLAP side.

5. Data Warehouse — The Organized Library

A **data warehouse** is a large, central database built specifically for **analytics (OLAP)**. It gathers data from many different operational systems, cleans and structures it, and stores it in a consistent, query-friendly format so analysts can ask questions across the whole business.

> **Analogy — A well-run library.** Books (data) arrive from many publishers (source systems) in different shapes and languages. Librarians clean them, translate them into one language, label them with a consistent system, and shelve them by category. Now anyone can walk in and find exactly what they need in minutes. A data warehouse does this for data: everything is cleaned, standardized, and neatly organized **before** it's stored.

> **Key trait — schema-on-write.** A warehouse decides the structure of the data **before** loading it. Data must be cleaned and fit a defined table shape (a *schema*) on the way in. This makes the data highly trustworthy and fast to query — but less flexible for messy or unusual data.

How a warehouse is organized: the star schema

Warehouses commonly use a **star schema**: one central **fact table** (the measurable events, e.g., sales) surrounded by **dimension tables** (the descriptive context: who, what, where, when). It's called a "star" because the diagram looks like a fact table in the middle with dimensions radiating out.

sale_id	date_key	product_key	store_key	units	revenue
9001	20260622	P-01	S-12	3	$13.50
9002	20260622	P-04	S-07	1	$5.00
9003	20260623	P-01	S-31	2	$9.00

Table 5.1 — `fact_sales` : the numbers we measure. IDs point to dimension tables.

product_key	name	category	size
P-01	Latte	Hot Coffee	Medium
P-04	Cold Brew	Cold Coffee	Large

Table 5.2 — `dim_product` : descriptive context the fact table refers to.

By joining the fact table to dimensions, an analyst can ask "revenue by **category** per **day**" without storing "Hot Coffee" on every one of a billion sales rows. Clean, efficient, and easy to explore.

Strengths and limits

Strengths	Limitations
Very fast analytical queries	Mostly handles structured (table) data
Clean, consistent, trustworthy data	Rigid — schema must be defined up front

Great for BI dashboards & reports	Storing huge raw volumes can be costly
Strong governance & security	Not ideal for images, video, free text, logs

Table 5.3 — Data warehouse at a glance.

Examples: **Snowflake, Amazon Redshift, Google BigQuery, Azure Synapse, Teradata.**

6. Data Lake — The Giant Reservoir

A **data lake** is a huge, low-cost storage repository that holds **any kind of data in its raw form** — structured tables, semi-structured JSON and CSV files, and unstructured images, audio, video, and logs. You pour data in first and decide how to use it later.

> **Analogy — A giant natural lake.** Rivers, rain, and streams (all your data sources) flow in freely, in whatever form they arrive. Nothing is filtered or bottled on the way in. When you actually need water, you take some out and treat it for its specific purpose. A data lake stores everything cheaply and raw; you clean it only when you use it.

> **Key trait — schema-on-read.** The opposite of a warehouse. A lake stores data **as-is**, and you apply a structure only **when you read it**. This is flexible and cheap, but without discipline a lake can become a "data swamp" — a disorganized dump nobody trusts.

Sample contents of a data lake

Unlike a warehouse's neat tables, a lake is really a big file store (often folders of files in cloud storage). Its contents are varied:

Path / object	Type	Example use
/raw/orders/2026-07-22.json	Semi-structured	Daily order exports from the app
/raw/clickstream/*.log	Unstructured logs	Website behavior analysis
/raw/reviews/*.csv	Structured text	Customer sentiment analysis
/raw/store_photos/*.jpg	Unstructured images	Computer-vision shelf checks

Table 6.1 — A data lake holds many data types side by side, raw.

> **Why data lakes exist:** Modern data isn't just neat tables. Machine learning needs images, text, and sensor logs. Warehouses struggle with those, and storing everything in a warehouse is expensive. A data lake stores **everything, cheaply**, so nothing is thrown away before its value is known.

Strengths and limits

Strengths	Limitations
Stores any data type	Easy to become a messy "data swamp"
Very cheap storage at scale	Weak data quality & governance by default
Flexible — no schema needed up front	Slower/harder to query than a warehouse
Great for ML & data science	No built-in transactions or reliability

Table 6.2 — Data lake at a glance.

Built on cloud object storage: **Amazon S3, Azure Data Lake Storage, Google Cloud Storage.**

Warehouse vs Lake — quick contrast

Aspect	Data Warehouse	Data Lake
Data type	Structured, cleaned	Any: raw, mixed
Schema	On write (before load)	On read (when used)
Cost	Higher	Lower
Quality	High, governed	Varies; needs discipline
Best for	BI & reporting	Data science & ML, raw archive

Table 6.3 — Two philosophies of storing analytical data.

7. Lakehouse — Best of Both Worlds

For years, companies had to run **both** a data lake (cheap, flexible, for raw data and ML) **and** a data warehouse (clean, fast, for BI). That meant two systems, two copies of data, and lots of extra pipelines. The **lakehouse** is a newer architecture that merges them: it adds the reliability and structure of a warehouse **directly on top of** the cheap, flexible storage of a lake.

> **Analogy — A reservoir with a modern treatment plant built in.** You still get the big, cheap reservoir that holds any kind of water (the lake). But now a high-quality treatment and bottling plant sits right on top, so you can also draw clean, reliable, drinkable water on demand (the warehouse) — all from one place, without hauling water to a separate facility.

> **The secret ingredient — the open table format.** Technologies like **Delta Lake, Apache Iceberg,** and **Apache Hudi** add a "transaction layer" over plain files in a lake. This layer gives lake files warehouse-like superpowers: **ACID transactions, schema enforcement, updates/deletes, and time travel** (querying data as it looked last week).

What the lakehouse gives you

- **One system, one copy of data** — no separate lake and warehouse to keep in sync.
- **Any data type** — like a lake (images, JSON, logs, tables).
- **Reliable transactions & quality** — like a warehouse.
- **Supports both BI dashboards and machine learning** from the same storage.
- **Time travel** — roll back mistakes or audit how data changed.

Feature	Data Warehouse	Data Lake	Lakehouse
Handles raw/unstructured data	No	Yes	Yes
Cheap storage	No	Yes	Yes
ACID transactions & reliability	Yes	No	Yes
Fast BI queries	Yes	Limited	Yes
Good for ML	Limited	Yes	Yes
Number of systems to run	Two (with a lake)	Two (with a warehouse)	One

Table 7.1 — Three generations of analytical storage.

Examples: **Databricks Lakehouse** (Delta Lake), **Snowflake** with Iceberg tables, **Apache Iceberg** on any cloud. The lakehouse is where much of the industry is heading, and it's the natural home for the Medallion Architecture we cover in Section 12.

8. ETL — Extract, Transform, Load

We have sources (OLTP systems, apps, files) and destinations (warehouses, lakes). **How does data actually get from one to the other, cleaned along the way?** The classic answer is **ETL**: **E**xtract, **T**ransform, **L**oad.

> **The three steps:**
> **Extract** — pull raw data out of source systems (databases, APIs, files).
> **Transform** — clean, reshape, combine, and validate the data *before* it lands.
> **Load** — write the finished, clean data into the destination (usually a warehouse).

> **Analogy — A restaurant kitchen. Extract** = collect raw ingredients from the market. **Transform** = wash, chop, cook, and plate them in the kitchen. **Load** = serve the finished dish to the diner. The customer only ever sees the clean, ready-to-eat result — never the raw, muddy vegetables. In ETL, the warehouse only receives fully prepared data.

A worked example

BrewBox wants clean daily sales in its warehouse. Raw data from the app is messy:

id	product	amount	city	ts
1	latte	4.5	london	1657872842
2	LATTE	4.5	London	1657872845
3	Cold Brew	NULL	LONDON	1657872849

Table 8.1 — EXTRACTED raw data (notice the problems).

Problems: product name capitalization is inconsistent, a price is missing, the city spelling varies, and the timestamp is a hard-to-read number. The **Transform** step fixes all of this:

id	product	amount	city	sale_time
1	Latte	$4.50	London	2026-07-15 09:34
2	Latte	$4.50	London	2026-07-15 09:34
3	Cold Brew	$5.00	London	2026-07-15 09:34

Table 8.2 — After TRANSFORM: standardized, cleaned, ready to load.

Names are title-cased and unified, the missing price is filled from a product price list, city spelling is standardized, and the timestamp is converted to a readable date. Only **then** is the clean table **Loaded** into the warehouse.

> **The defining trait of ETL:** data is **transformed BEFORE it is loaded**. The destination only ever holds clean data. This was the standard for decades, especially when storage and computing were expensive and you didn't want to waste them on raw junk.

9. ELT — Extract, Load, Transform

ELT is the modern reordering of ETL: **E**xtract, **L**oad, **T**ransform. You extract raw data and **load it immediately** into a powerful destination (a cloud warehouse, lake, or lakehouse) — and **then transform it there**, using the destination's own massive computing power.

> **Analogy — A meal-kit delivery service.** Instead of a chef cooking everything before it reaches you (ETL), the company ships all the raw ingredients straight to your kitchen (Extract → Load), and you cook exactly the dishes you want, when you want them (Transform, later, on demand). You keep the raw ingredients too, so you can cook something different tomorrow without re-ordering.

Why ELT became popular

Two things changed: cloud storage became **very cheap**, and cloud warehouses/lakehouses became **incredibly powerful**. So it now makes sense to dump raw data in first and transform it in place.

- **Speed to land data** — raw data arrives fast; no waiting on transforms first.
- **Keep the raw data** — because you stored it before transforming, you can re-transform it differently later for new needs (great for ML).
- **Scales effortlessly** — the warehouse/lakehouse does the heavy transformation with its huge compute.

ETL vs ELT — side by side

Aspect	ETL	ELT
Order	Extract → Transform → Load	Extract → Load → Transform
Where transform happens	In a separate tool, before loading	Inside the destination, after loading
Destination gets	Only clean data	Raw data first, then cleaned in place
Raw data kept?	Usually discarded	Yes — stored and reusable
Best fit	On-premise, limited compute, strict pre-cleaning	Cloud warehouses & lakehouses
Flexibility	Lower (fixed transforms up front)	Higher (re-transform anytime)
Typical tools	Informatica, Talend, SSIS	dbt, Fivetran + Snowflake/BigQuery

Table 9.1 — Same three letters, different order, big difference.

> **How to remember it: ETL** cleans the ingredients *before* they enter the kitchen; **ELT** brings all ingredients into a big modern kitchen and cooks them *there*. ELT dominates today because cloud storage is cheap and cloud compute is strong — but ETL is still common where data must be cleaned or masked (e.g., for privacy) before it lands anywhere.

10. Apache Spark — The Engine That Crunches Big Data

When data is small, one computer can process it. But when you have **billions of rows** — far more than one machine's memory or CPU can handle — you need many computers working together. **Apache Spark** is the most popular open-source engine for processing huge datasets **in parallel across a cluster of machines**.

> **Analogy — Counting votes in an election.** One person counting 10 million ballots alone would take weeks. Instead, you split the ballots across 1,000 volunteers who each count their pile at the same time (in parallel), then combine the subtotals. Spark is the organizer that splits the work, hands pieces to many machines, and merges the results. The many machines together are called a **cluster**.

> **Key idea — distributed, in-memory processing.** Spark splits a big job into small tasks, runs them on many machines **at the same time**, and keeps data **in memory (RAM)** where possible instead of slow disk. That combination makes it fast for massive data.

How Spark divides the work

A Spark cluster has a **driver** (the manager that plans the job) and many **executors** (the workers that do the counting). Big data is split into chunks called **partitions**, and each executor processes its partitions independently.

Worker (executor)	Partition handled	Rows	Local subtotal (revenue)
Executor 1	Orders A–F	250 million	$1.12 B
Executor 2	Orders G–M	250 million	$0.98 B
Executor 3	Orders N–S	250 million	$1.05 B
Executor 4	Orders T–Z	250 million	$1.01 B
Driver combines subtotals →			**$4.16 B total**

Table 10.1 — A billion-row job, split across a cluster.

Each worker sums its own slice at the same time; the driver just adds four subtotals at the end. Four machines finish in roughly a quarter of the time one machine would take.

What Spark is used for

- **Batch processing** — transform enormous datasets (the "T" in big-data ETL/ELT).
- **Streaming** — process data continuously as it arrives (Spark Structured Streaming).
- **Machine learning** — train models on large data (Spark MLlib).
- **SQL analytics** — run SQL queries over huge tables (Spark SQL).

> **Where you'll meet it:** Spark is the engine under platforms like **Databricks** and **Amazon EMR**. When you hear "we process our data with Spark," it means the transformation and heavy lifting in the

> pipeline runs on a Spark cluster. It's the workhorse that powers the transform steps and the Medallion layers ahead.

11. Data Pipeline — Moving Data From A to B

A **data pipeline** is the end-to-end series of automated steps that moves data from its sources, through processing, to its destination — **reliably and on a schedule**, without a human doing it by hand each time. ETL and ELT are *patterns* for pipelines; Spark is often the *engine* inside them.

> **Analogy — A factory conveyor belt.** Raw materials go in one end. Along the belt, each station does one job: inspect, wash, assemble, paint, package. At the far end, a finished product rolls off — automatically, the same way every time. A data pipeline is that conveyor belt for data: each stage does one transformation, and the whole thing runs on its own.

Stages of a typical pipeline

Stage	What happens	Example
1. Ingest	Pull data from sources	Export yesterday's orders from the OLTP database & app API
2. Store (raw)	Land raw data cheaply	Drop JSON files into the data lake
3. Process	Clean & transform (often Spark)	Standardize products, fix prices, join with store info
4. Serve	Load into warehouse/lakehouse	Write clean sales into analytics tables
5. Consume	Dashboards, reports, ML use it	Manager opens the daily sales dashboard

Table 11.1 — BrewBox's daily sales pipeline, stage by stage.

Batch vs streaming pipelines

Aspect	Batch pipeline	Streaming pipeline
When it runs	On a schedule (e.g., every night)	Continuously, as data arrives
Data handled	Big chunks at once	Small events one by one
Freshness	Hours old	Seconds old
Example	Nightly sales report	Live fraud detection on payments

Table 11.2 — Two ways a pipeline can run.

> **Orchestration.** Pipelines have many steps that must run in the right order, on schedule, with retries if something fails. Tools called **orchestrators** — such as **Apache Airflow, Dagster,** or **Prefect** — act as the conductor, starting each step at the right time and alerting engineers when something breaks.

> **Why pipelines matter:** A good pipeline runs automatically, handles failures gracefully, and delivers **fresh, trustworthy data** every day without anyone babysitting it. Building and maintaining reliable pipelines is the day-to-day heart of a data engineer's job.

12. Medallion Architecture — Bronze, Silver, Gold

Inside a lakehouse, how do you keep raw data *and* clean data *and* business-ready data organized? The **Medallion Architecture** is a simple, popular design (championed by Databricks) that organizes data into **three progressively cleaner layers**, named after medals: **Bronze** → **Silver** → **Gold**. Data flows through them, getting more refined and more valuable at each step.

> **Analogy — Refining gold ore. Bronze** is the raw ore straight from the mine — dirty, mixed with rock, but complete. **Silver** is the ore after washing and removing impurities — cleaner and usable. **Gold** is the polished, finished jewelry ready for the shop window. Each stage adds value; you never skip straight from mine to jewelry.

The three layers

Bronze — Raw

Data lands here exactly as it came from the source, untouched. It's the historical record. If anything goes wrong later, you can always reprocess from Bronze.

id	prod	amt	city	ts	_ingested_at
1	latte	4.5	london	1657872842	2026-07-22 02:00
2	LATTE	4.5	London	1657872845	2026-07-22 02:00
3	Cold Brew	NULL	LONDON	1657872849	2026-07-22 02:00

Table 12.1 — BRONZE: raw ingested orders, warts and all.

Silver — Cleaned & Conformed

Here the data is cleaned, deduplicated, validated, and joined with other sources. It's reliable and consistent — the trustworthy "single source of truth" that data scientists and analysts build on.

order_id	product	amount	city	region	sale_time
1	Latte	$4.50	London	North	2026-07-15 09:34
3	Cold Brew	$5.00	London	North	2026-07-15 09:34

Table 12.2 — SILVER: cleaned, standardized, enriched with store & region.

(The duplicate row 2 was removed; the missing price was filled; the timestamp was made readable; region was added.)

Gold — Business-Ready

The top layer: aggregated, summarized data shaped for a specific business purpose — dashboards, KPIs, reports, or ML features. This is what decision-makers actually consume.

date	region	total_units	total_revenue
2026-07-15	North	18,204	$86,918

2026-07-15	South	21,530	$99,145

Table 12.3 — GOLD: aggregated daily revenue by region, ready for the dashboard.

Why teams love it

Layer	State of data	Who uses it	Purpose
Bronze	Raw, unfiltered	Data engineers	Complete history; safe to reprocess
Silver	Clean, validated	Engineers, data scientists	Trusted single source of truth
Gold	Aggregated, business-shaped	Analysts, managers, BI, ML	Fast, ready-to-use insights

Table 12.4 — What each layer is for.

- **Clarity** — everyone knows what quality to expect at each layer.
- **Safety** — if a bug corrupts Silver or Gold, you rebuild from untouched Bronze.
- **Reusability** — one clean Silver layer feeds many different Gold tables.
- **Incremental quality** — problems are caught and fixed step by step, not all at once.

13. How It All Fits Together

Let's trace one cup of coffee's data journey through everything you've learned, end to end.

Step	What happens	Concept in action
1	A customer buys a latte; the order is saved instantly.	**OLTP** database (e.g., PostgreSQL)
2	Overnight, an automated job copies all orders out.	**Data pipeline** — Ingest / Extract
3	Raw orders land untouched in cheap cloud storage.	**Data lake** → **Bronze** layer
4	A cluster cleans, dedupes, and enriches the data.	**Apache Spark** → **Silver** layer
5	Clean data is loaded, then aggregated in place.	**ELT** in a **Lakehouse** → **Gold** layer
6	A manager opens the daily revenue dashboard.	**OLAP** analytics on the **Gold** table

Table 13.1 — One latte, from checkout to boardroom.

Every concept in this guide plays a role in that single journey:

- **OLTP** captured the sale, live and accurate.
- A **pipeline** moved the data automatically, on schedule.
- The **data lake** / **lakehouse** stored it cheaply and flexibly.
- **Medallion (Bronze→Silver→Gold)** refined it step by step.
- **Spark** did the heavy processing across many machines.
- **ELT** loaded raw first, then transformed in place.
- **OLAP** turned it all into an answer a human could act on.

> **The one-sentence summary of data engineering:** take raw data from the systems that *run* the business (OLTP), move it reliably through pipelines, store and refine it (lake → lakehouse, Bronze → Silver → Gold) using powerful engines like Spark, and deliver clean, ready answers to the systems that help you *understand* the business (OLAP).

What to learn next

Now that the concepts make sense, good next steps for a fresher are: **SQL** (the essential language for all of this), a cloud platform (**AWS, Azure,** or **GCP**), **Python** for scripting, hands-on **PySpark** on **Databricks'** free Community Edition, and an orchestrator like **Apache Airflow**. Build a small project — even moving a CSV through a Bronze→Silver→Gold pipeline — and you'll have applied every idea in this guide.

14. Common Questions from Freshers (FAQ)

These are the questions that come up again and again — in study groups and in interviews. Short, clear answers you can rely on.

Is a database the same as a data warehouse?

No. A "database" usually means an **OLTP** database that runs an application (fast, single-record operations). A **data warehouse** is a special kind of database tuned for **OLAP** — big analytical reads across lots of history. Same family, very different jobs.

Data lake or data warehouse — which should a company use?

Often **both**, or increasingly a **lakehouse** that combines them. Use a warehouse when you mainly need clean structured data for dashboards; use a lake when you also have raw, mixed, or huge data for data science. The lakehouse exists so you don't have to choose.

ETL or ELT — which is "better"?

Neither is universally better; it depends on context. **ELT** is the modern default on cloud platforms because storage is cheap and compute is powerful. **ETL** still wins when data must be cleaned, filtered, or masked for privacy *before* it's allowed to land anywhere.

Do I need to know how to code to be a data engineer?

Yes, but you can start small. **SQL** is non-negotiable — it's the language of nearly every tool here. **Python** is the most common general-purpose language for pipelines and Spark (via PySpark). You don't need to be a computer-science expert on day one; you grow into it.

Is Apache Spark always necessary?

No. Spark shines on **big data** — datasets too large for one machine. For small or medium data, a plain database or a single Python/pandas job is simpler and cheaper. Reach for Spark when the data genuinely outgrows one computer.

What's the difference between batch and real-time?

Batch processes data in scheduled chunks (e.g., every night) — simple and common. **Real-time** / **streaming** processes each event within seconds — needed for things like fraud detection or live dashboards, but more complex to build and run.

Where does the Medallion Architecture actually live?

Usually inside a **lakehouse**. Bronze, Silver, and Gold are just three sets of tables (or folders) at increasing levels of cleanliness. It's a naming and organizing convention, not a separate product you install.

> **Interview tip:** If you can confidently explain the OLTP→OLAP journey and where each concept (pipeline, lake, Spark, ELT, Medallion) fits into it — using a simple example like the coffee shop — you'll handle most fresher-level data-engineering interviews with ease.

15. Roles & Skills in Data Engineering

Data engineering sits alongside a few related roles. Knowing who does what helps you find where you fit and who you'll work with.

Role	Main focus	Typical output
Data Engineer	Build & run pipelines; move and shape data	Reliable, clean data tables ready to use
Data Analyst	Explore data; answer business questions	Dashboards, reports, insights
Data Scientist	Build statistical / ML models	Predictions, models, experiments
ML Engineer	Put models into production	Deployed, monitored ML systems
Analytics Engineer	Transform data (ELT) for analysts	Clean, modelled warehouse tables

Table 15.1 — Data engineer vs neighbouring roles.

Think of it as a relay: the **data engineer** lays the track and delivers clean data; analysts, data scientists, and ML engineers then run with it to produce insights and products.

Core skills to build, in order

Priority	Skill	Why it matters
1	**SQL**	The universal language of data — used by every tool in this guide.
2	**Python**	Scripting pipelines, automation, and PySpark.
3	**A cloud platform** (AWS / Azure / GCP)	Modern data lives in the cloud; storage & compute run there.
4	**Data modelling** (star schema, Medallion)	Structuring data so it's efficient and easy to query.
5	**Apache Spark** (PySpark)	Processing data too big for one machine.
6	**Orchestration** (Airflow)	Scheduling and reliably running multi-step pipelines.

Table 15.2 — A practical learning path for a fresher.

The tools landscape — a cheat sheet

Job posts throw a lot of tool names at you. Here's how the popular ones map onto the concepts in this guide, so the buzzwords stop being scary.

Job in the pipeline	Concept	Popular tools you'll hear about
Run the app / capture data	OLTP	PostgreSQL, MySQL, Oracle, MongoDB
Cheap raw storage	Data Lake	Amazon S3, Azure Data Lake, Google Cloud Storage
Analytics warehouse	Data Warehouse / OLAP	Snowflake, BigQuery, Redshift, Synapse
Unified lake + warehouse	Lakehouse	Databricks, Delta Lake, Apache Iceberg

Move data in (ingest)	Extract / Load	Fivetran, Airbyte, Kafka (streaming)
Transform data	Transform (ELT/ETL)	dbt, Apache Spark, SQL
Heavy big-data processing	Distributed compute	Apache Spark (on Databricks / EMR)
Schedule & coordinate	Orchestration	Apache Airflow, Dagster, Prefect
Show the results	BI / OLAP consumption	Power BI, Tableau, Looker

Table 15.3 — Common tools grouped by the job they do.

You do not need to learn all of these. Master the concept, learn one tool per row, and you can pick up the rest quickly — because they all do the same underlying job.

> **A first project to try.** Take any public CSV (say, city weather or sales data). Load it raw into a folder (**Bronze**), clean it with Python or SQL into a tidy table (**Silver**), then produce a small summary such as "average by month" (**Gold**). Congratulations — you've just built a Medallion pipeline and touched most concepts in this guide.

16. Glossary of Key Terms

Term	Plain-English meaning
OLTP	Online Transaction Processing — systems that handle fast, live, everyday transactions that run the business.
OLAP	Online Analytical Processing — systems built to analyze large amounts of historical data for insights.
ACID	Atomicity, Consistency, Isolation, Durability — guarantees that keep transactions correct and reliable.
Data Warehouse	A central, structured database for analytics; data is cleaned before loading (schema-on-write).
Data Lake	Cheap storage for any raw data type; structure applied when read (schema-on-read).
Lakehouse	Combines a lake's flexibility and low cost with a warehouse's reliability and structure.
Schema	The defined structure of data — its columns, types, and rules.
Star Schema	Warehouse design with a central fact table surrounded by dimension tables.
Fact / **Dimension**	Fact = the numbers you measure (sales); Dimension = descriptive context (product, store, date).
ETL	Extract, Transform, Load — clean data *before* loading it to the destination.
ELT	Extract, Load, Transform — load raw data first, then transform it inside the destination.
Apache Spark	An engine that processes huge datasets in parallel across many machines (a cluster).
Cluster	A group of machines working together as one to process data.
Partition	A chunk of a big dataset that one worker processes independently.
Data Pipeline	Automated, scheduled steps that move data from source to destination reliably.
Batch / **Streaming**	Batch = process data in scheduled chunks; Streaming = process events continuously as they arrive.
Orchestrator	Tool (Airflow, Dagster, Prefect) that schedules and coordinates pipeline steps.
Medallion Architecture	Organizing data into Bronze (raw) → Silver (clean) → Gold (business-ready) layers.
Time Travel	Querying data as it looked at an earlier point in time (a lakehouse feature).
BI	Business Intelligence — dashboards and reports that turn data into decisions.

Table 14.1 — Quick reference for every term in this guide.

Congratulations! You now understand the core vocabulary and architecture of modern data engineering — from the tiny transaction at a coffee counter all the way to the executive dashboard. Keep this guide handy as a reference, and start building.